

Pac-Man Project Documentation

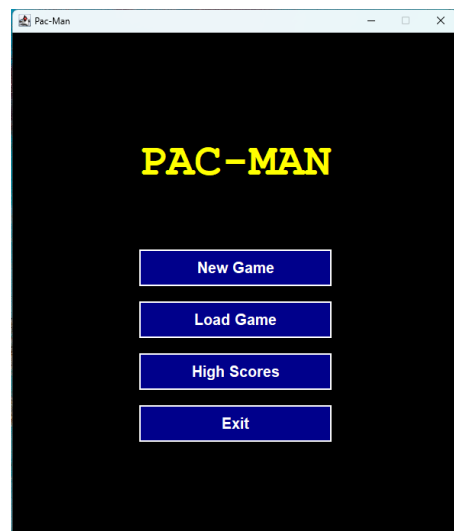
1. Project Overview

1.1 Purpose of the Program

The goal of this project is to implement a classic Pac-Man style game using **Java** with a **Swing** graphical user interface. In the game, the user controls a yellow character in a maze, aiming to collect all points while avoiding ghosts. The software supports saving and loading game states, as well as maintaining a High Score list for competitive play.

1.2 Functional Requirements (Use-Cases)

- **Start New Game:** Loads the default map and initializes Pac-Man and ghosts to their starting positions.
- **Movement and Control:** The player controls the protagonist using arrow keys; the program continuously monitors for wall collisions.
- **Scoring:** Collecting pellets on the map increases the player's score.
- **Save and Load Game:** The current game state (positions, lives, score) can be saved to and loaded from a **JSON** file.
- **Leaderboard:** Displays the top 10 results in a tabular format.



2. User Manual

2.1 Starting the Game

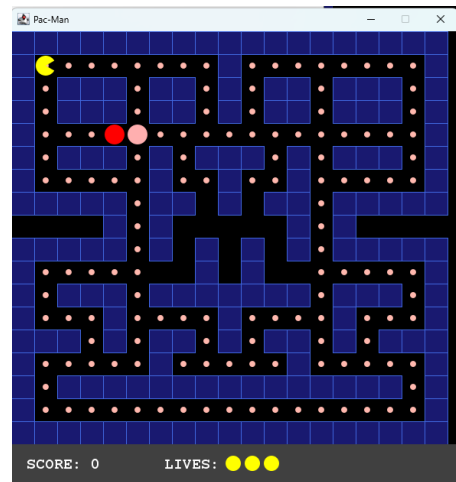
Upon launching, a modern, button-based **Main Menu** appears. The available options are:

- **New Game:** Starts a fresh game with 0 points and 3 lives.

- **Load Game:** Resumes the last session from the saved_game.json file.
- **High Scores:** Lists previous best results.
- **Exit:** Closes the application.

2.2 Controls and Rules

- **Arrow Keys:** Move Pac-Man (Up, Down, Left, Right).
- **ESC Key:** Pause the game and display the Pause menu.
- **Gameplay Rules:**
 1. The player starts with **3 lives**.
 2. Each collected pellet is worth **10 points**.
 3. Red and pink ghosts move towards the player. Sometimes they move randomly. Contact with a ghost results in losing a life and returning to the start position.
 4. The game ends when all lives are lost: **Game Over** or all pellets are collected (**You Win**).



HIGH SCORES		
RANK	NAME	SCORE
1	LACI	1390
2	LILI	1380
3	JÁNOS	630
4	PÉTER	0

BACK TO MENU

3. Design and Implementation

3.1 Technology Stack

The application was developed in a **Java 17** environment. The UI is built with **Java Swing**, utilizing the **Graphics2D** class for rendering, enabling features like antialiasing and dynamic shape drawing (e.g., Pac-Man's animated mouth).

3.2 Architecture

The software follows the **"Model-View-Separation"** principle, organized into three main packages for maintainability:

- **hu.pacman.model:** Contains game logic, rules, and state (no GUI code).
- **hu.pacman.gui:** UI elements including the main window, menus, and game panel.
- **hu.pacman.data:** Data persistence layer for file I/O.

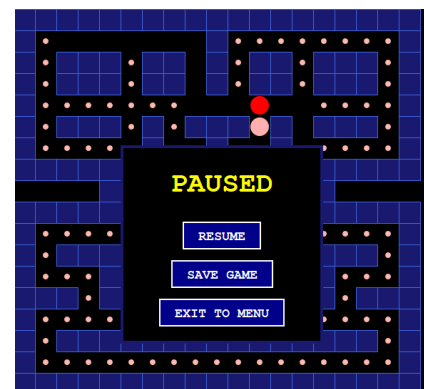
3.3 Detailed Class Descriptions

- **Game:** The central controller that manages the game loop, keyboard events, and state updates.
- **Map:** Represents the level using a 2D integer array (0: empty, 1: wall, 2: pellet).
- **Player:** Stores Pac-Man's data and implements collision detection (tryMove).
- **Ghost:** Determines the player's position to attempt movement toward them; moves randomly if a wall is hit.
- **ScoreManager:** Contains static methods for handling scores.json.

4. Data Structures and Algorithms

4.1 Key Data Structures

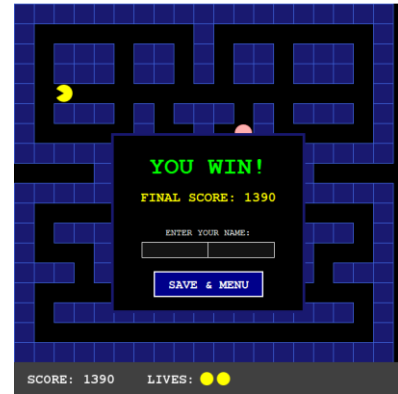
- **2D Array (int[][]):** Used for the map grid to provide fast access for collision checks.
- **ArrayList<Ghost>:** Stores ghosts dynamically, allowing for future expansion of the ghost count.
- **HashMap<String, Integer>:** Flexible storage for game metadata (score, lives) within the Game class.



4.2 Game Loop

The game uses a **javax.swing.Timer** running at 20ms intervals (approx. **50 FPS**).

- Player movement occurs every 5th cycle.
- Ghost movement occurs every 8th cycle, making ghosts slower than the player for balanced gameplay.



4.3 File Handling (JSON)

To minimize dependencies, the program uses custom string manipulation algorithms for JSON processing instead of external libraries.

- **Save Game:** Stores the map state and coordinates of all entities.
- **Leaderboard:** Appends the player's name and score to scores.json, sorted in descending order upon loading.

5. Testing

The project utilized **JUnit 5** to verify critical business logic:

- **PlayerTest:** Ensures walls block movement and pellet collection increases the score.
- **CollisionTest:** Validates life loss detection during player-ghost contact.
- **ScoreManagerTest:** Verifies correct file I/O and ensures the program handles corrupted files without crashing.

